

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

по Лабораторной работе № 5

«Процедуры, функции, триггеры в PostgreSQL»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Забродин Максим Александрович

Группа K3240

Направление подготовки 09.03.03 Прикладная информатика

Преподаватели: Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург

2026

Цель работы

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание

- Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР № 2).
- Создать триггеры для индивидуальной БД согласно варианту 2.2: 5 оригинальных (авторских) триггеров.

Индивидуальное задание (вариант)

Вариант 9. БД «Оптовая база»

Часть 4 индивидуального задания ЛР № 2 (процедуры):

- снижение цены на заданный процент для товаров, у которых срок пребывания на складе превысил заданный норматив;
- расчёт стоимости всех партий товаров, проданных за прошедшие сутки.

Третья процедура (авторская, в дополнение к указанным выше двум) реализует регистрацию оплаты счёта по поставке.

Часть 5 индивидуального задания ЛР № 2 — «Создать необходимые триггеры» — реализована согласно варианту 2.2 методических указаний к ЛР № 5: пять авторских триггеров, обеспечивающих целостность складского учёта, автоматизацию документооборота и контроль статусов заказов в БД «Оптовая база».

Выполнение

Подключение к серверу PostgreSQL

Работа выполнена в консоли SQL Shell. Подключение к ранее созданной базе данных «Оптовая база» выполнено командой:

```
psql -U postgres -h localhost -d opt_base
```

Листинг 1 — Подключение к базе данных opt_base через psql

После подключения схема поиска по умолчанию установлена командой SET search_path TO wholesale; — это позволяет обращаться к таблицам без указания имени схемы.

Часть 1. Хранимые процедуры (часть 4 индивидуального задания ЛР № 2)

Процедура 1. Снижение цены товаров с превышенным сроком хранения

Формулировка: снизить закупочную цену на заданный процент для партий товара, чей срок хранения на складе (от даты поставки до текущей даты) превысил заданный норматив в днях. Учитываются только партии с положительным остатком на складе.

Процедура использует IN-параметры (норматив срока хранения в днях и процент скидки), цикл FOR по результатам запроса и условные операторы для проверки корректности входных данных.

```
maksim@MacBook-Air-Karina-3 ~ % psql -U postgres -h localhost -d opt_base
psql (10.3 (Postgres.app))
Type "help" for help.

opt_base=# SET search_path TO wholesale;
SET
opt_base=# CREATE OR REPLACE PROCEDURE sp_discount_old_stock(
opt_base(#   IN p_days_threshold   INTEGER,
opt_base(#   IN p_discount_percent NUMERIC
opt_base(# )
opt_base=# LANGUAGE plpgsql
opt_base=# AS $$
opt_base=# DECLARE
opt_base=#   v_item RECORD;
opt_base=#   v_old_price NUMERIC(12,2);
opt_base=#   v_new_price NUMERIC(12,2);
opt_base=#   v_count   INTEGER := 0;
opt_base=# BEGIN
opt_base=#   IF p_days_threshold < 0 THEN
opt_base=#     RAISE EXCEPTION 'Норматив срока хранения не может быть отрицательным: %', p_days_threshold;
opt_base=#   END IF;
opt_base=#   IF p_discount_percent <= 0 OR p_discount_percent >= 100 THEN
opt_base=#     RAISE EXCEPTION 'Процент скидки должен быть в диапазоне (0;100): %', p_discount_percent;
opt_base=#   END IF;
opt_base=#   FOR v_item IN
opt_base=#     SELECT si.supply_item_id,
opt_base=#           si.purchase_unit_price,
opt_base=#           s.supply_date
opt_base=#     FROM supply_items si
opt_base=#     JOIN supplies s ON s.supply_id = si.supply_id
opt_base=#     WHERE si.remaining_stock_quantity > 0
opt_base=#           AND s.supply_date <= CURRENT_DATE - p_days_threshold
opt_base=#   LOOP
opt_base=#     v_old_price := v_item.purchase_unit_price;
opt_base=#     v_new_price := ROUND(v_old_price * (1 - p_discount_percent / 100.0), 2);
opt_base=#     UPDATE supply_items
opt_base=#     SET purchase_unit_price = v_new_price
opt_base=#     WHERE supply_item_id = v_item.supply_item_id;
opt_base=#     v_count := v_count + 1;
opt_base=#     RAISE NOTICE 'Партия supply_item_id = % : цена снижена с % до % (дата поставки %)',
opt_base=#       v_item.supply_item_id, v_old_price, v_new_price, v_item.supply_date;
opt_base=#   END LOOP;
opt_base=#   IF v_count = 0 THEN
opt_base=#     RAISE NOTICE 'Не найдено партий товара со сроком хранения более % дней.', p_days_threshold;
opt_base=#   ELSE
opt_base=#     RAISE NOTICE 'Снижена цена для % партий товара.', v_count;
opt_base=#   END IF;
opt_base=# END;
opt_base=# $$;
CREATE PROCEDURE
opt_base=#
```

Рисунок 2 — Создание процедуры sp_discount_old_stock в psql

Проверка работы процедуры на корректных данных. Перед вызовом просмотрено состояние таблицы supply_items:

```

opt_base=# SELECT si.supply_item_id, si.product_id, si.purchase_unit_price,
opt_base=#         si.remaining_stock_quantity, s.supply_date,
opt_base=#         CURRENT_DATE - s.supply_date AS days_in_stock
opt_base=# FROM supply_items si
opt_base=# JOIN supplies s ON s.supply_id = si.supply_id
opt_base=# ORDER BY si.supply_item_id;
 supply_item_id | product_id | purchase_unit_price | remaining_stock_quantity | supply_date | days_in_stock
-----
1 | 1 | 65000.00 | 15.000 | 2025-03-05 | 478
2 | 2 | 60.50 | 430.000 | 2025-03-07 | 476
3 | 3 | 1200.00 | 80.000 | 2025-03-10 | 473
4 | 2 | 60.50 | 90.000 | 2025-04-10 | 442
5 | 1 | 64000.00 | 8.000 | 2025-04-12 | 440
6 | 1 | 66000.00 | 6.000 | 2025-04-15 | 437
7 | 3 | 1250.00 | 45.000 | 2025-04-20 | 432
8 | 2 | 200.000 | 2026-06-25 | 1
9 | 3 | 1200.00 | 30.000 | 2026-06-20 | 6
(9 rows)

opt_base=#

```

Рисунок 3 — Состояние таблицы `supply_items` до вызова процедуры `sp_discount_old_stock`

Вызов процедуры с нормативом 90 дней и скидкой 10%:

```

opt_base=# CALL sp_discount_old_stock(90, 10);
NOTICE: Партия supply_item_id = 1 : цена снижена с 65000.00 до 58500.00 (дата поставки 2025-03-05)
NOTICE: Партия supply_item_id = 3 : цена снижена с 1200.00 до 1080.00 (дата поставки 2025-03-10)
NOTICE: Партия supply_item_id = 5 : цена снижена с 64000.00 до 57600.00 (дата поставки 2025-04-12)
NOTICE: Партия supply_item_id = 6 : цена снижена с 66000.00 до 59400.00 (дата поставки 2025-04-15)
NOTICE: Партия supply_item_id = 7 : цена снижена с 1250.00 до 1125.00 (дата поставки 2025-04-20)
NOTICE: Партия supply_item_id = 2 : цена снижена с 60.50 до 54.45 (дата поставки 2025-03-07)
NOTICE: Партия supply_item_id = 4 : цена снижена с 60.50 до 54.45 (дата поставки 2025-04-10)
NOTICE: Снижена цена для 7 партий товара.
CALL
opt_base=#

```

Рисунок 4 — Результат вызова `sp_discount_old_stock(90, 10)` — снижена цена для 7 партий товара

Процедура корректно обработала 7 партий товара (`supply_item_id` от 1 до 7), поставленных более 90 дней назад, и снизила цену на 10% для каждой из них. Партии 8 и 9, поставленные недавно (1 и 6 дней назад соответственно), под действие скидки не попали, что подтверждает корректность условия отбора по сроку хранения.

```

opt_base=# SELECT si.supply_item_id, si.product_id, si.purchase_unit_price,
opt_base=#         s.supply_date, CURRENT_DATE - s.supply_date AS days_in_stock
opt_base=# FROM supply_items si
opt_base=# JOIN supplies s ON s.supply_id = si.supply_id
opt_base=# ORDER BY si.supply_item_id;
 supply_item_id | product_id | purchase_unit_price | supply_date | days_in_stock
-----+-----+-----+-----+-----
              1 |          1 |         58500.00 | 2025-03-05 |           478
              2 |          2 |           54.45 | 2025-03-07 |           476
              3 |          3 |         1080.00 | 2025-03-10 |           473
              4 |          2 |           54.45 | 2025-04-10 |           442
              5 |          1 |        57600.00 | 2025-04-12 |           440
              6 |          1 |        59400.00 | 2025-04-15 |           437
              7 |          3 |         1125.00 | 2025-04-20 |           432
              8 |          2 |           60.50 | 2026-06-25 |             1
              9 |          3 |         1200.00 | 2026-06-20 |             6
(9 rows)

opt_base=# █

```

Рисунок 5 — Состояние таблицы `supply_items` после вызова процедуры — цены партий 1–7 снижены на 10%, цены партий 8 и 9 не изменились

Проверка работы процедуры на некорректных входных данных:

```

opt_base=# CALL sp_discount_old_stock(90, 150);
ERROR:  Процент скидки должен быть в диапазоне (0;100): 150
CONTEXT:  PL/pgSQL function sp_discount_old_stock(integer,numeric) line 13 at RAISE
opt_base=# █

```

Рисунок 6 — Ошибка при вызове `sp_discount_old_stock(90, 150)` — процент скидки вне допустимого диапазона

```

opt_base=# CALL sp_discount_old_stock(-5, 10);
ERROR:  Норматив срока хранения не может быть отрицательным: -5
CONTEXT:  PL/pgSQL function sp_discount_old_stock(integer,numeric) line 9 at RAISE
opt_base=# █

```

Рисунок 7 — Ошибка при вызове `sp_discount_old_stock(-5, 10)` — отрицательный норматив срока хранения

В обоих случаях процедура отклонила некорректные входные данные с информативным сообщением об ошибке, не внося изменений в таблицу.

Процедура 2. Расчёт стоимости партий товаров, проданных за сутки

Формулировка: рассчитать общую стоимость (сумму продаж) всех партий товаров, проданных за указанные сутки. Поскольку в таблице `order_items` отсутствует собственная

дата заказа, в качестве даты продажи партии используется дата поставки (supplies.supply_date) — это единственная доступная в структуре БД дата, связанная с конкретной проданной партией товара. Процедура использует IN-параметр (дата) и OUT-параметр для возврата рассчитанной суммы.

```
opt_base=# CREATE OR REPLACE PROCEDURE sp_daily_sales_total(
opt_base(#      IN p_sale_date      DATE,
opt_base(#      OUT p_total_amount NUMERIC
opt_base(# )
opt_base=# LANGUAGE plpgsql
opt_base=# AS $$
opt_base$# BEGIN
opt_base$#     SELECT COALESCE(SUM(oi.ordered_quantity * oi.sale_unit_price), 0)
opt_base$#     INTO p_total_amount
opt_base$#     FROM order_items oi
opt_base$#     JOIN supply_items si ON si.supply_id = oi.supply_id AND si.product_id = oi.product_id
opt_base$#     JOIN supplies s ON s.supply_id = si.supply_id
opt_base$#     WHERE s.supply_date = p_sale_date;
opt_base$#
opt_base$#     RAISE NOTICE 'Стоимость партий товаров, проданных за %, составила % руб.',
opt_base$#           p_sale_date, p_total_amount;
opt_base$# END;
opt_base$# $$;
opt_base=# CREATE PROCEDURE
opt_base=#
```

Рисунок 8 — Создание процедуры *sp_daily_sales_total* в *psql*

Проверка процедуры на дату, за которую в БД зафиксированы продажи:

```
opt_base=# CALL sp_daily_sales_total('2026-06-25', NULL);
NOTICE:  Стоимость партий товаров, проданных за 2026-06-25, составила 325.00000 руб.
p_total_amount
-----
325.00000
```

Рисунок 9 — Результат *sp_daily_sales_total('2026-06-25', NULL)* — сумма продаж 325.00 руб.

За 25 июня 2026 года была продана единственная партия — 5 единиц сахара по цене 65.00 руб., что даёт сумму $5 \times 65.00 = 325.00$ руб. — результат процедуры подтверждён.

Проверка процедуры на дату, за которую продаж не было:

```

opt_base=# CALL sp_daily_sales_total('2026-01-01', NULL);
NOTICE: Стоимость партий товаров, проданных за 2026-01-01, составила 0 руб.
 p_total_amount
-----
          0
(1 row)

opt_base=# █

```

Рисунок 10 — Результат `sp_daily_sales_total('2026-01-01', NULL)` — корректный результат 0 благодаря COALESCE

Благодаря применению функции COALESCE процедура корректно возвращает 0 при отсутствии продаж за указанные сутки, а не NULL и не ошибку выполнения.

Процедура 3 (авторская). Регистрация оплаты счёта по поставке

Формулировка: перевести счёт поставки (таблица `supply_invoices`) в статус «Paid» и проставить дату оплаты текущей датой. Процедура использует IN-параметр (идентификатор поставки) и INOUT-параметр, через который возвращается текстовое описание результата операции: успешная оплата, попытка повторной оплаты уже оплаченного счёта, попытка оплаты отменённого счёта или отсутствие счёта по указанной поставке.


```

opt_base=# CREATE OR REPLACE PROCEDURE sp_register_supply_payment(
opt_base(#      IN      p_supply_id INTEGER,
opt_base(#      INOUT   p_result  VARCHAR
opt_base(# )
opt_base=# LANGUAGE plpgsql
opt_base=# AS $$
opt_base$# DECLARE
opt_base$#     v_invoice_id      INTEGER;
opt_base$#     v_invoice_status  VARCHAR(30);
opt_base$# BEGIN
opt_base$#     SELECT si.invoice_id, si.invoice_status
opt_base$#     INTO v_invoice_id, v_invoice_status
opt_base$#     FROM supply_invoices si
opt_base$#     WHERE si.supply_id = p_supply_id;
opt_base$#
opt_base$#     IF NOT FOUND THEN
opt_base$#         p_result := FORMAT('Счёт по поставке %s не найден.', p_supply_id);
opt_base$#         RAISE NOTICE '%', p_result;
opt_base$#         RETURN;
opt_base$#     END IF;
opt_base$#
opt_base$#     IF v_invoice_status = 'Paid' THEN
opt_base$#         p_result := FORMAT('Счёт по поставке %s уже оплачен.', p_supply_id);
opt_base$#         RAISE NOTICE '%', p_result;
opt_base$#         RETURN;
opt_base$#     END IF;
opt_base$#
opt_base$#     IF v_invoice_status = 'Cancelled' THEN
opt_base$#         p_result := FORMAT('Счёт по поставке %s отменён, оплата невозможна.', p_supply_id);
opt_base$#         RAISE NOTICE '%', p_result;
opt_base$#         RETURN;
opt_base$#     END IF;
opt_base$#
opt_base$#     UPDATE supply_invoices
opt_base$#     SET invoice_status = 'Paid',
opt_base$#         payment_date = CURRENT_DATE
opt_base$#     WHERE supply_id = p_supply_id;
opt_base$#
opt_base$#     p_result := FORMAT('Счёт по поставке %s успешно оплачен %s.', p_supply_id, CURRENT_DATE);
opt_base$#     RAISE NOTICE '%', p_result;
opt_base$# END;
opt_base=# $$;
CREATE PROCEDURE
opt_base=#

```

Рисунок 11 — Создание процедуры `sp_register_supply_payment` в `psql`

Состояние таблицы `supply_invoices` перед тестированием процедуры (видны неоплаченные счета в статусе «Created» по поставкам 3, 7 и 9):

```

opt_base=# SELECT * FROM supply_invoices ORDER BY invoice_id;

```

invoice_id	supply_id	invoice_amount	invoice_status	creation_date	payment_date
1	1	130000.00	Paid	2025-03-05	2025-03-06
2	2	27500.00	Paid	2025-03-07	2025-03-08
3	3	120000.00	Created	2025-03-10	
4	4	5500.00	Paid	2025-04-10	2025-04-11
5	5	640000.00	Paid	2025-04-12	2025-04-13
6	6	528000.00	Paid	2025-04-15	2025-04-16
7	7	62500.00	Created	2025-04-20	
8	8	11000.00	Paid	2026-06-25	2026-06-25
9	9	36000.00	Created	2026-06-20	

```

(9 rows)

opt_base=#

```


Рисунок 12 — Состояние таблицы *supply_invoices* до вызова процедуры *sp_register_supply_payment*

Корректный вызов — оплата неоплаченного счёта по поставке 7:

```
[opt_base=# CALL sp_register_supply_payment(7, NULL);
NOTICE: Счёт по поставке 7 успешно оплачен 2026-06-26.
        p_result
-----
 Счёт по поставке 7 успешно оплачен 2026-06-26.
(1 row)

opt_base=# █
```

Рисунок 13 — Результат успешной оплаты счёта по поставке 7 — *INOUT*-параметр *p_result* содержит сообщение об успехе

```
[opt_base=# CALL sp_register_supply_payment(7, NULL);
NOTICE: Счёт по поставке 7 успешно оплачен 2026-06-26.
        p_result
-----
 Счёт по поставке 7 успешно оплачен 2026-06-26.
(1 row)

[opt_base=# SELECT * FROM supply_invoices WHERE supply_id = 7;
 invoice_id | supply_id | invoice_amount | invoice_status | creation_date | payment_date
-----+-----+-----+-----+-----+-----
          7 |          7 |       62500.00 | Paid           | 2025-04-20   | 2026-06-26
(1 row)

opt_base=# █
```

Рисунок 14 — Счёт по поставке 7 переведён в статус *Paid*, дата оплаты проставлена текущей датой

Проверка защиты от повторной оплаты — попытка оплатить уже оплаченный счёт:

```
[opt_base=# CALL sp_register_supply_payment(7, NULL);
NOTICE:  Счёт по поставке 7 уже оплачен.
          p_result
-----
 Счёт по поставке 7 уже оплачен.
(1 row)

opt_base=# █
```

Рисунок 15 — Повторная оплата отклонена — счёт уже оплачен

Проверка обработки несуществующей поставки:

```
[opt_base=# CALL sp_register_supply_payment(999, NULL);
NOTICE:  Счёт по поставке 999 не найден.
          p_result
-----
 Счёт по поставке 999 не найден.
(1 row)

opt_base=# █
```

Рисунок 16 — Процедура корректно сообщает об отсутствии счёта по несуществующей поставке

Во всех трёх «узких местах» процедура не выбрасывает необработанное исключение и не изменяет данные, а возвращает понятное текстовое описание ситуации через INOUT-параметр, что удобно для последующей обработки результата вызывающим кодом.

Часть 2. Триггеры (вариант 2.2 — пять авторских триггеров)

В соответствии с вариантом 2.2 методических указаний разработано пять оригинальных триггеров, реализующих контроль целостности складского учёта, автоматизацию документооборота по поставкам и контроль корректности изменения статусов заказов в БД «Оптовая база».

Триггер 1. Контроль остатков партии товара (supply_items)

Назначение: при добавлении новой партии товара (INSERT), если остаток на складе (remaining_stock_quantity) не указан явно (NULL), он автоматически приравнивается к поставленному количеству (supplied_quantity) — вся поставленная партия изначально находится на складе. Дополнительно при INSERT и UPDATE проверяется, что остаток не превышает поставленное количество и не является отрицательным — это дублирующая проверка целостности данных (помимо ограничения CHECK на таблице), учитывающая случай изменения остатка триггером.

```
Счёт по поставке / уже оплачен.
(1 row)

opt_base=# CALL sp_register_supply_payment(999, NULL);
NOTICE: Счёт по поставке 999 не найден.
P_result

-----
Счёт по поставке 999 не найден.
(1 row)

opt_base=# clear
opt_base=# CREATE OR REPLACE FUNCTION fn_check_supply_items_stock()
opt_base=# RETURNS TRIGGER AS $$
opt_base$# BEGIN
opt_base$#     IF TG_OP = 'INSERT' AND NEW.remaining_stock_quantity IS NULL THEN
opt_base$#         NEW.remaining_stock_quantity := NEW.supplied_quantity;
opt_base$#     END IF;
opt_base$#     IF NEW.remaining_stock_quantity > NEW.supplied_quantity THEN
opt_base$#         RAISE EXCEPTION
opt_base$#             'Остаток партии (%) не может превышать поставленное количество (%).',
opt_base$#             NEW.remaining_stock_quantity, NEW.supplied_quantity;
opt_base$#     END IF;
opt_base$#     IF NEW.remaining_stock_quantity < 0 THEN
opt_base$#         RAISE EXCEPTION 'Остаток партии не может быть отрицательным: %', NEW.remaining_stock_quantity;
opt_base$#     END IF;
opt_base$#     RETURN NEW;
opt_base$# END;
opt_base$# $$ LANGUAGE plpgsql;
ERROR: syntax error at or near "clear"
LINE 1: clear
        ^

opt_base=#
opt_base=# CREATE TRIGGER trg_check_supply_items_stock
opt_base=# BEFORE INSERT OR UPDATE ON supply_items
opt_base=# FOR EACH ROW EXECUTE PROCEDURE fn_check_supply_items_stock();
ERROR: function fn_check_supply_items_stock() does not exist
opt_base=#
```

Рисунок 17 — Ошибочный ввод команды clear внутри блока определения функции — функция и триггер не созданы

После повторного, корректного выполнения скрипта функция и триггер были успешно созданы:

```
opt_base=# CREATE OR REPLACE FUNCTION fn_check_supply_items_stock()
opt_base=# RETURNS TRIGGER AS $$
opt_base$# BEGIN
opt_base$#     IF TG_OP = 'INSERT' AND NEW.remaining_stock_quantity IS NULL THEN
opt_base$#         NEW.remaining_stock_quantity := NEW.supplied_quantity;
opt_base$#     END IF;
opt_base$#     IF NEW.remaining_stock_quantity > NEW.supplied_quantity THEN
opt_base$#         RAISE EXCEPTION
opt_base$#             'Остаток партии (%) не может превышать поставленное количество (%).',
opt_base$#             NEW.remaining_stock_quantity, NEW.supplied_quantity;
opt_base$#     END IF;
opt_base$#     IF NEW.remaining_stock_quantity < 0 THEN
opt_base$#         RAISE EXCEPTION 'Остаток партии не может быть отрицательным: %', NEW.remaining_stock_quantity;
opt_base$#     END IF;
opt_base$#     RETURN NEW;
opt_base$# END;
opt_base$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
opt_base=#
```

Рисунок 18 — Успешное создание функции `fn_check_supply_items_stock`

```
opt_base=# CREATE TRIGGER trg_check_supply_items_stock
opt_base=# BEFORE INSERT OR UPDATE ON supply_items
opt_base=# FOR EACH ROW EXECUTE PROCEDURE fn_check_supply_items_stock();
CREATE TRIGGER
opt_base=#
```

Рисунок 19 — Успешное создание триггера `trg_check_supply_items_stock`

Проверка на корректных данных — добавление новой партии без явного указания остатка:

```
opt_base=# INSERT INTO supply_items (supply_id, product_id, batch_number, supplied_quantity, purchase_unit_price, remaining_stock_quantity)
opt_base=# VALUES (1, 1, 'BATCH-LAP-004', 5.000, 67000.00, NULL);
INSERT 0 1
opt_base=# SELECT * FROM supply_items WHERE batch_number = 'BATCH-LAP-004';
 supply_item_id | supply_id | product_id | batch_number | supplied_quantity | purchase_unit_price | remaining_stock_quantity
-----
(1 row)
10 | 1 | 1 | BATCH-LAP-004 | 5.000 | 67000.00 | 5.000
opt_base=#
```

Рисунок 20 — Остаток партии `BATCH-LAP-004` автоматически установлен равным `supplied_quantity = 5.000`

Проверка на некорректных данных — попытка указать остаток больше поставленного количества:

```
opt_base=# INSERT INTO supply_items (supply_id, product_id, batch_number, supplied_quantity, purchase_unit_price, remaining_stock_quantity)
opt_base=# VALUES (1, 1, 'BATCH-LAP-005', 5.000, 67000.00, 10.000);
ERROR: Остаток партии (10.000) не может превышать поставленное количество (5.000).
CONTEXT: PL/pgSQL function fn_check_supply_items_stock() line 8 at RAISE
opt_base=#
```

Рисунок 21 — Триггер отклонил вставку: остаток (10.000) превышает поставленное количество (5.000)

Триггер 2. Списание остатка склада при оформлении позиции заказа (`order_items`)

Назначение: при добавлении новой позиции заказа (INSERT в `order_items`) триггер проверяет, что в соответствующей партии товара на складе (`supply_items`) достаточно остатка для заказываемого количества, и если достаточно — автоматически списывает заказанное количество с остатка партии. Если партия не найдена или остатка недостаточно, вставка отклоняется с исключением, а остаток не изменяется.

```

opt_base=# CREATE OR REPLACE FUNCTION fn_reserve_stock_on_order()
opt_base=# RETURNS TRIGGER AS $$
opt_base=# DECLARE
opt_base=#     v_remaining NUMERIC(12,3);
opt_base=# BEGIN
opt_base=#     SELECT remaining_stock_quantity INTO v_remaining
opt_base=#     FROM supply_items
opt_base=#     WHERE supply_id = NEW.supply_id
opt_base=#           AND product_id = NEW.product_id
opt_base=#           AND batch_number = NEW.batch_number
opt_base=#     FOR UPDATE;
opt_base=#
opt_base=#     IF NOT FOUND THEN
opt_base=#         RAISE EXCEPTION
opt_base=#         'Партия (supply_id = %, product_id = %, batch_number = %) не найдена на складе.',
opt_base=#         NEW.supply_id, NEW.product_id, NEW.batch_number;
opt_base=#     END IF;
opt_base=#
opt_base=#     IF v_remaining < NEW.ordered_quantity THEN
opt_base=#         RAISE EXCEPTION
opt_base=#         'Недостаточно товара на складе: остаток % меньше заказанного количества %.',
opt_base=#         v_remaining, NEW.ordered_quantity;
opt_base=#     END IF;
opt_base=#
opt_base=#     UPDATE supply_items
opt_base=#     SET remaining_stock_quantity = remaining_stock_quantity - NEW.ordered_quantity
opt_base=#     WHERE supply_id = NEW.supply_id
opt_base=#           AND product_id = NEW.product_id
opt_base=#           AND batch_number = NEW.batch_number;
opt_base=#
opt_base=#     RETURN NEW;
opt_base=# END;
opt_base=# $$ LANGUAGE plpgsql;
CREATE FUNCTION
opt_base=#
opt_base=# CREATE TRIGGER trg_reserve_stock_on_order
opt_base=# BEFORE INSERT ON order_items
opt_base=# FOR EACH ROW EXECUTE PROCEDURE fn_reserve_stock_on_order();
CREATE TRIGGER
opt_base=#

```

Рисунок 22 — Успешное создание функции fn_reserve_stock_on_order и триггера trg_reserve_stock_on_order

Состояние остатка партии BATCH-LAP-004 перед оформлением заказа:

```

opt_base=# SELECT * FROM supply_items WHERE batch_number = 'BATCH-LAP-004';
 supply_item_id | supply_id | product_id | batch_number | supplied_quantity | purchase_unit_price | remaining_stock_quantity
-----
(1 row)

```

supply_item_id	supply_id	product_id	batch_number	supplied_quantity	purchase_unit_price	remaining_stock_quantity
10	1	1	BATCH-LAP-004	5.000	67000.00	5.000

```

opt_base=#

```

Рисунок 23 — Остаток партии BATCH-LAP-004 составляет 5.000 единиц

Проверка на корректных данных — оформление позиции заказа на 3 единицы товара:

```

opt_base=# INSERT INTO order_items (supply_id, product_id, batch_number, ordered_quantity, sale_unit_price)
opt_base=# VALUES (1, 1, 'BATCH-LAP-004', 3.000, 72000.00);
INSERT 0 1
opt_base=# SELECT * FROM supply_items WHERE batch_number = 'BATCH-LAP-004';
 supply_item_id | supply_id | product_id | batch_number | supplied_quantity | purchase_unit_price | remaining_stock_quantity
-----
(1 row)

```

supply_item_id	supply_id	product_id	batch_number	supplied_quantity	purchase_unit_price	remaining_stock_quantity
10	1	1	BATCH-LAP-004	5.000	67000.00	2.000

```

opt_base=#

```


*Рисунок 24 — Остаток партии автоматически уменьшен с 5.000 до 2.000 (5 минус 3)
после добавления позиции заказа*

Проверка на некорректных данных — заказ количества, превышающего остаток на складе:

```
opt_base=# INSERT INTO order_items (supply_id, product_id, batch_number, ordered_quantity, sale_unit_price)
opt_base=# VALUES (1, 1, 'BATCH-LAP-004', 100.000, 72000.00);
ERROR: Недостаточно товара на складе: остаток 2.000 меньше заказанного количества 100.000.
CONTEXT: PL/pgSQL function fn_reserve_stock_on_order() line 19 at RAISE
opt_base=# █
```

*Рисунок 25 — Триггер отклонил вставку: остаток (2.000) меньше заказанного
количества (100.000)*

Проверка на некорректных данных — заказ по несуществующей партии товара:

```
opt_base=# INSERT INTO order_items (supply_id, product_id, batch_number, ordered_quantity, sale_unit_price)
opt_base=# VALUES (1, 1, 'BATCH-NOT-EXIST', 1.000, 72000.00);
ERROR: Партия (supply_id = 1, product_id = 1, batch_number = BATCH-NOT-EXIST) не найдена на складе.
CONTEXT: PL/pgSQL function fn_reserve_stock_on_order() line 13 at RAISE
opt_base=# █
```

Рисунок 26 — Триггер отклонил вставку: партия BATCH-NOT-EXIST не найдена на складе

Триггер 3. Автоматическое создание счёта по поставке (supplies)

Назначение: при изменении статуса поставки на «Received» (AFTER UPDATE) триггер автоматически создаёт счёт на оплату поставки (supply_invoices), если счёт для этой поставки ещё не существует. Сумма счёта рассчитывается как сумма произведений поставленного количества на закупочную цену по всем позициям поставки (supply_items).


```

opt_base=# CREATE OR REPLACE FUNCTION fn_autocreate_supply_invoice()
opt_base=# RETURNS TRIGGER AS $$
opt_base=# DECLARE
opt_base=#     v_amount NUMERIC(12,2);
opt_base=# BEGIN
opt_base=#     IF NEW.status = 'Received' AND (OLD.status IS DISTINCT FROM 'Received') THEN
opt_base=#         IF EXISTS (SELECT 1 FROM supply_invoices WHERE supply_id = NEW.supply_id) THEN
opt_base=#             RETURN NEW;
opt_base=#         END IF;
opt_base=#         SELECT COALESCE(SUM(supplied_quantity * purchase_unit_price), 0)
opt_base=#         INTO v_amount
opt_base=#         FROM supply_items
opt_base=#         WHERE supply_id = NEW.supply_id;
opt_base=#         IF v_amount > 0 THEN
opt_base=#             INSERT INTO supply_invoices
opt_base=#             (supply_id, invoice_amount, invoice_status, creation_date, payment_date)
opt_base=#             VALUES
opt_base=#             (NEW.supply_id, v_amount, 'Created', CURRENT_DATE, NULL);
opt_base=#             RAISE NOTICE 'Создан счёт на сумму % по поставке %.', v_amount, NEW.supply_id;
opt_base=#         END IF;
opt_base=#     END IF;
opt_base=#     RETURN NEW;
opt_base=# END;
opt_base=# $$ LANGUAGE plpgsql;
opt_base=# CREATE FUNCTION
opt_base=#
opt_base=# CREATE TRIGGER trg_autocreate_supply_invoice
opt_base=# AFTER UPDATE ON supplies
opt_base=# FOR EACH ROW EXECUTE PROCEDURE fn_autocreate_supply_invoice();
opt_base=# CREATE TRIGGER
opt_base=#

```

Рисунок 27 — Успешное создание функции fn_autocreate_supply_invoice и триггера trg_autocreate_supply_invoice

Проверка работы триггера. Сначала создана тестовая поставка со статусом «Planned» (без счёта):

```

opt_base=# INSERT INTO supplies (supplier_id, contract_number, invoice_id, supply_date, status, description, waybill_number, actual_supply_date)
opt_base=# VALUES (1, 'SUP-2025-001', NULL, CURRENT_DATE, 'Planned', 'Тестовая поставка для проверки триггера', 'WB-010', NULL);
opt_base=# INSERT 0 1
opt_base=#

```

Рисунок 28 — Тестовая поставка WB-010 создана со статусом Planned

Для поставки добавлена позиция товара (необходима для расчёта суммы будущего счёта):

```

opt_base=# INSERT INTO supply_items (supply_id, product_id, batch_number, supplied_quantity, purchase_unit_price, remaining_stock_quantity)
opt_base=# VALUES (
opt_base=#     (SELECT supply_id FROM supplies WHERE waybill_number = 'WB-010'),
opt_base=#     1, 'BATCH-LAP-006', 4.000, 67000.00, NULL
opt_base=# );
opt_base=# INSERT 0 1
opt_base=#

```

Рисунок 29 — Позиция партии BATCH-LAP-006 добавлена для поставки WB-010

Проверено, что счёт по поставке пока не создан:

```

opt_base=# INSERT INTO supply_items (supply_id, product_id, batch_number, supplied_quantity, purchase_unit_price, remaining_stock_quantity)
opt_base=# VALUES (
opt_base=# (SELECT supply_id FROM supplies WHERE waybill_number = 'WB-010'),
opt_base=# 1, 'BATCH-LAP-006', 4.000, 67000.00, NULL
opt_base=# );
ERROR: duplicate key value violates unique constraint "uq_supply_items_batch"
DETAIL: Key (supply_id, product_id, batch_number)=(10, 1, BATCH-LAP-006) already exists.
opt_base=# SELECT * FROM supply_invoices
opt_base=# WHERE supply_id = (SELECT supply_id FROM supplies WHERE waybill_number = 'WB-010');
 invoice_id | supply_id | invoice_amount | invoice_status | creation_date | payment_date
-----+-----+-----+-----+-----+-----
(0 rows)

opt_base=# █

```

Рисунок 30 — Счёт по поставке WB-010 отсутствует (0 строк) — подтверждено перед сменой статуса

Изменение статуса поставки на «Received» — срабатывание триггера:

```

opt_base=# UPDATE supplies
opt_base=# SET status = 'Received', actual_supply_date = CURRENT_DATE
opt_base=# WHERE waybill_number = 'WB-010';
NOTICE: Создан счёт на сумму 268000.00 по поставке 10.
UPDATE 1
opt_base=# █

```

Рисунок 31 — Триггер автоматически создал счёт на сумму 268000.00 руб. (4 на 67000.00) по поставке

```

opt_base=# SELECT * FROM supply_invoices
opt_base=# WHERE supply_id = (SELECT supply_id FROM supplies WHERE waybill_number = 'WB-010');
 invoice_id | supply_id | invoice_amount | invoice_status | creation_date | payment_date
-----+-----+-----+-----+-----+-----
10 | 10 | 268000.00 | Created | 2026-06-26 |
(1 row)

opt_base=# █

```

Рисунок 32 — Автоматически созданный счёт: сумма 268000.00 руб., статус Created, дата создания — текущая дата

Триггер 4. Контроль корректности изменения статуса заказа (orders)

Назначение: запретить некорректные («регрессивные») переходы статуса заказа.

Допустимая последовательность статусов: Created -> Paid -> Assembled -> Picked up. Из любого статуса, кроме финальных (Picked up, Cancelled), допускается переход в статус Cancelled. Изменение статуса заказа, уже находящегося в финальном статусе (Picked up или Cancelled), запрещено.

```

opt_base=# CREATE OR REPLACE FUNCTION fn_check_order_status_transition()
opt_base=# RETURNS TRIGGER AS $$
opt_base=# DECLARE
opt_base=#     v_old_rank INTEGER;
opt_base=#     v_new_rank INTEGER;
opt_base=# BEGIN
opt_base=#     IF NEW.order_status = OLD.order_status THEN
opt_base=#         RETURN NEW;
opt_base=#     END IF;
opt_base=#
opt_base=#     IF OLD.order_status IN ('Picked up', 'Cancelled') THEN
opt_base=#         RAISE EXCEPTION
opt_base=#             'Невозможно изменить статус заказа % : текущий статус "%" является финальным.',
opt_base=#             OLD.order_id, OLD.order_status;
opt_base=#     END IF;
opt_base=#
opt_base=#     IF NEW.order_status = 'Cancelled' THEN
opt_base=#         RETURN NEW;
opt_base=#     END IF;
opt_base=#
opt_base=#     v_old_rank := CASE OLD.order_status
opt_base=#                     WHEN 'Created' THEN 1
opt_base=#                     WHEN 'Paid' THEN 2
opt_base=#                     WHEN 'Assembled' THEN 3
opt_base=#                     ELSE NULL
opt_base=#                 END;
opt_base=#
opt_base=#     v_new_rank := CASE NEW.order_status
opt_base=#                     WHEN 'Created' THEN 1
opt_base=#                     WHEN 'Paid' THEN 2
opt_base=#                     WHEN 'Assembled' THEN 3
opt_base=#                     WHEN 'Picked up' THEN 4
opt_base=#                     ELSE NULL
opt_base=#                 END;
opt_base=#
opt_base=#     IF v_old_rank IS NULL OR v_new_rank IS NULL OR v_new_rank <= v_old_rank THEN
opt_base=#         RAISE EXCEPTION
opt_base=#             'Недопустимый переход статуса заказа % : "%" -> "%".',
opt_base=#             OLD.order_id, OLD.order_status, NEW.order_status;
opt_base=#     END IF;
opt_base=#
opt_base=#     RETURN NEW;
opt_base=# END;
opt_base=# $$ LANGUAGE plpgsql;
opt_base=#
opt_base=# CREATE FUNCTION
opt_base=#
opt_base=# CREATE TRIGGER trg_check_order_status_transition
opt_base=# BEFORE UPDATE ON orders
opt_base=# FOR EACH ROW EXECUTE PROCEDURE fn_check_order_status_transition();
opt_base=#
opt_base=# CREATE TRIGGER
opt_base=#

```

Рисунок 33 — Успешное создание функции `fn_check_order_status_transition` и триггера `trg_check_order_status_transition`

Текущие статусы заказов перед тестированием триггера:

```
[opt_base=# SELECT order_id, order_status FROM orders ORDER BY order_id;
 order_id | order_status
-----+-----
         1 | Paid
         2 | Created
         3 | Assembled
         4 | Paid
         5 | Paid
         6 | Picked up
         7 | Picked up
         8 | Picked up
(8 rows)

opt_base=#
```

Рисунок 34 — Заказ 2 находится в статусе Created, заказ 6 — в финальном статусе Picked up

Проверка на корректных данных — допустимый переход Created -> Paid:

```
[opt_base=# UPDATE orders SET order_status = 'Paid' WHERE order_id = 2;
UPDATE 1
opt_base=#
```

Рисунок 35 — Переход Created -> Paid выполнен без ошибок

Проверка на некорректных данных — попытка "откатить" статус назад:

```
[opt_base=# UPDATE orders SET order_status = 'Created' WHERE order_id = 2;
ERROR: Недопустимый переход статуса заказа 2 : "Paid" -> "Created".
CONTEXT: PL/pgSQL function fn_check_order_status_transition() line 36 at RAISE
opt_base=#
```

Рисунок 36 — Триггер отклонил переход "Paid" -> "Created" как недопустимый

Проверка защиты финального статуса — попытка изменить статус уже выданного заказа:

```
[opt_base=# UPDATE orders SET order_status = 'Paid' WHERE order_id = 6;
ERROR: Невозможно изменить статус заказа 6 : текущий статус "Picked up" является финальным.
CONTEXT: PL/pgSQL function fn_check_order_status_transition() line 11 at RAISE
opt_base=#
```

Рисунок 37 — Триггер отклонил изменение статуса заказа 6: текущий статус Picked up является финальным

Проверка на корректных данных — отмена заказа из промежуточного статуса (разрешена в любой момент, кроме финальных статусов):

Команда выполнена успешно (UPDATE 1) — отмена заказа из промежуточного статуса Paid разрешена логикой триггера независимо от рангов статусов.

Триггер 5. Журнал истории изменения статусов заказов (orders)

Назначение: вести журнал истории изменения статусов заказов в отдельной таблице order_status_log. При создании нового заказа (AFTER INSERT) в журнал записывается его начальный статус (старое значение — NULL); при изменении статуса существующего заказа (AFTER UPDATE) — переход «старый статус -> новый статус». Журнал ведётся независимо от триггера 4 — поскольку оба триггера определены для таблицы orders, но один из них BEFORE, а другой AFTER, PostgreSQL гарантированно выполняет проверку корректности перехода (триггер 4) раньше записи в журнал (триггер 5): если переход недопустим, выполнение прерывается исключением до срабатывания AFTER-триггера, и ошибочная запись в журнал не попадает.


```

opt_base=# CREATE TABLE IF NOT EXISTS order_status_log (
opt_base(#      log_id      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
opt_base(#      order_id   INTEGER NOT NULL,
opt_base(#      old_status  VARCHAR(30),
opt_base(#      new_status  VARCHAR(30) NOT NULL,
opt_base(#      changed_at  TIMESTAMP NOT NULL DEFAULT NOW(),
opt_base(#
opt_base(#      CONSTRAINT fk_order_status_log_orders
opt_base(#          FOREIGN KEY (order_id)
opt_base(#          REFERENCES orders(order_id)
opt_base(#          ON UPDATE CASCADE
opt_base(#          ON DELETE CASCADE
opt_base(# );
CREATE TABLE
opt_base=#
opt_base=# CREATE OR REPLACE FUNCTION fn_log_order_status()
opt_base=# RETURNS TRIGGER AS $$
opt_base$# BEGIN
opt_base$#     IF TG_OP = 'INSERT' THEN
opt_base$#         INSERT INTO order_status_log (order_id, old_status, new_status)
opt_base$#         VALUES (NEW.order_id, NULL, NEW.order_status);
opt_base$#         RETURN NEW;
opt_base$#
opt_base$#     ELSIF TG_OP = 'UPDATE' THEN
opt_base$#         IF NEW.order_status IS DISTINCT FROM OLD.order_status THEN
opt_base$#             INSERT INTO order_status_log (order_id, old_status, new_status)
opt_base$#             VALUES (NEW.order_id, OLD.order_status, NEW.order_status);
opt_base$#         END IF;
opt_base$#         RETURN NEW;
opt_base$#     END IF;
opt_base$#
opt_base$#     RETURN NEW;
opt_base$# END;
opt_base$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
opt_base=#
opt_base=# CREATE TRIGGER trg_log_order_status
opt_base=# AFTER INSERT OR UPDATE ON orders
opt_base=# FOR EACH ROW EXECUTE PROCEDURE fn_log_order_status();
CREATE TRIGGER
opt_base=# █

```

Рисунок 38 — Успешное создание таблицы `order_status_log`, функции `fn_log_order_status` и триггера `trg_log_order_status`

Проверка на событии INSERT — создание нового заказа:

```

opt_base=# INSERT INTO orders (order_status, customer_id, contract_number, order_date)
opt_base=# VALUES ('Created', 1, 'SALE-2025-001', CURRENT_DATE);
INSERT 0 1
opt_base=# SELECT * FROM order_status_log ORDER BY log_id;
 log_id | order_id | old_status | new_status |          changed_at
-----+-----+-----+-----+-----
      1 |       10 |          | Created   | 2026-06-26 15:44:15.122186
(1 row)

opt_base=# █

```


Рисунок 39 — При создании заказа (*order_id* = 10) в журнал автоматически добавлена запись: *old_status* = NULL, *new_status* = Created

Проверка на событии UPDATE — изменение статуса заказа:

```
opt_base=# UPDATE orders
opt_base=# SET order_status = 'Paid'
[opt_base=# WHERE order_id = 10;
UPDATE 1
[opt_base=# SELECT * FROM order_status_log ORDER BY log_id;
 log_id | order_id | old_status | new_status |          changed_at
-----+-----+-----+-----+-----
      1 |      10 |          | Created   | 2026-06-26 15:44:15.122186
      2 |      10 | Created   | Paid      | 2026-06-26 15:44:40.038151
(2 rows)

opt_base=#
```

Рисунок 40 — Журнал зафиксировал переход *old_status* = Created, *new_status* = Paid для заказа 10

Журнал корректно фиксирует как создание заказа, так и каждое последующее изменение его статуса, что подтверждает правильность работы триггера.

Выводы

В ходе выполнения лабораторной работы № 5 получены практические навыки создания и использования хранимых процедур, функций и триггеров в СУБД PostgreSQL средствами консоли SQL Shell (psql), применительно к индивидуальной БД «Оптовая база» (вариант 9).

Разработаны три хранимые процедуры в соответствии с частью 4 индивидуального задания ЛР № 2: процедура снижения закупочной цены товаров с превышенным сроком хранения (с использованием IN-параметров и цикла FOR по результатам запроса), процедура расчёта стоимости партий товаров, проданных за заданные сутки (с использованием OUT-параметра), и авторская процедура регистрации оплаты счёта по поставке (с использованием INOUT-параметра для возврата текстового результата операции). Все три процедуры протестированы как на корректных, так и на заведомо

некорректных входных данных; во всех случаях обработка ошибок выполнена средствами RAISE EXCEPTION либо через информативный результат в выходном параметре.

Разработаны пять авторских триггеров (вариант 2.2 индивидуального задания), охватывающих ключевые бизнес-процессы оптовой базы: контроль остатков партии товара на складе с автоматическим заполнением и проверкой целостности данных; автоматическое списание остатка склада при оформлении позиции заказа с защитой от превышения доступного количества; автоматическое формирование счёта по поставке при получении товара; контроль допустимых переходов статуса заказа, предотвращающий некорректные («регрессивные») изменения и изменение заказов в финальном статусе; ведение журнала истории изменения статусов заказов. Для каждого триггера продемонстрирована работа как на корректных, так и на ошибочных данных, выявлены и зафиксированы «узкие места» обработки (недостаточный остаток товара, несуществующая партия, попытка повторной оплаты счёта, попытка некорректного изменения статуса заказа).

Таким образом, все пункты практического задания лабораторной работы выполнены в полном объёме: созданы и протестированы три хранимые процедуры и пять авторских триггеров для индивидуальной БД «Оптовая база».